

Backend Engineering Launchpad – Case Study

Draftly: Gmail AI Reply Agent

Author : Airtribe

Draftly

BACKGROUND

Professionals spend hours each week drafting routine email replies such as confirmations, follow-ups, meeting responses, or polite acknowledgements. Automating these repetitive tasks can improve productivity and ensure consistency.

This project will design and implement a backend for an AI-powered Gmail assistant that can generate and manage reply drafts on the user's behalf, while giving the user full control before anything is sent.

OBJECTIVE

Build a reliable and extensible Gmail AI agent that can:

- Fetch incoming emails from a user's Gmail inbox
- Generate draft replies using an AI model
- Learn from the user's past email patterns to better match tone, phrasing, and style
- Allow review and approval before sending
- Send approved replies via Gmail with proper threading and metadata

KEY FEATURES

Gmail Integration

- Connect securely using Gmail API with OAuth2
- Fetch unread or recent emails with metadata such as sender, subject, body, timestamp, and thread ID

Draft Generation

- Use an AI model to generate reply drafts
- Support multiple tones such as formal, concise, or friendly
- Insert context from the existing thread and user preferences such as signature
- Learn from user's last few sent emails to infer style, tone, and phrasing preferences

Draft Review and Approval

- Provide APIs for users to view suggested drafts, approve, edit, or reject them
- Only approved drafts can be sent

Reply Sending

- Send approved drafts via Gmail API
- Maintain thread integrity by using correct message IDs and headers

Logging and History

- Store each draft and final sent email
- Track draft status such as approved, edited, or rejected
- Provide logs for monitoring and debugging

Failure Handling

- Retry failed sends automatically
- Notify the user if persistent errors occur such as expired token

Authentication and Security

- Implement OAuth2 flow for Gmail access
- Encrypt stored tokens and user preferences
- Provide logout and token revocation support

TECHNICAL REQUIREMENTS

- Expose all functionality as RESTful APIs.
- Use a database to store drafts, logs, and user preferences.
- Integrate with the Gmail API for fetching and sending.
- Implement idempotency and retries for sending.
- Ensure compliance with Gmail API quotas and security requirements.

ASSESSMENT CRITERIA

- **Functionality:** Does the system work as intended? Does it meet all the requirements stated above?
- **Code Quality:** Is your code clean, organized, well-commented, and following best practices?
- **Design and Structure:** Is the system well-designed? Does it demonstrate a good understanding of system design principles and patterns?
- **Documentation:** Is your report comprehensive and clear? Does it effectively explain the choices made and how to use the project?
- **Presentation:** Do you effectively demonstrate and explain your system and the decisions made during its development?

DELIVERABLES

1. The final, functional product.
2. README file outlining how to use the system, API documentation, the design decisions and other necessary information.
3. Public link of the Github repository
4. Explainer video demonstrating your project work